Trabajo Práctico Integrador – Bases de Datos II

Tecnicatura Universitaria en Programación – UTN

Integrantes: Lucas Matías Catanzaro, Emiliano Oscar Fiscina.

1. Descripción del proyecto elegido

El dominio elegido es una clínica veterinaria**.** La clínica atiende a dueños de mascotas (clientes), registra a los animales que se atienden (mascotas) y agenda las consultas (turnos).
El problema que se busca resolver es que, cuando esta información se lleva en papel o en planillas sueltas, es fácil perder el historial de un animal, duplicar datos de un mismo cliente o no saber qué turnos están pendientes. Se propone centralizar todo en una base de datos en la nube, accesible desde una aplicación, con un historial que nunca se pierde.
Se eligió este dominio porque las relaciones entre sus entidades son naturales y permiten justificar con claridad las decisiones de modelado: un cliente tiene muchas mascotas y una mascota tiene muchos turnos.

2. Modelo de datos

El modelo tiene tres colecciones relacionadas, dentro de la base tpi_veterinaria

Colección	Contenido
Clientes	Dueños de las mascotas, con su dirección
Mascotas	Animales atendidos, con sus vacunas, vinculados a un cliente
Turnos	Consultas agendadas, vinculadas a una mascota

Esquemas JSON

```
// clientes
{
  "_id": 1,
  "nombre": "Laura Pérez",
  "email": "laura@mail.com",
  "telefono": "2262-555555",
  "direccion": { "calle": "Calle 4 N° 1234", "ciudad": "Necochea" },
  "activo": true,
"fechaAlta": { "$date": "2026-10-07T16:20:00Z" }
}
// mascotas
{
  "_id": 1,
  "clienteId": 1,
  "nombre": "Firulais",
  "especie": "Perro",
  "raza": "Mestizo",
"vacunas": [ { "nombre": "Antirrábica", "fecha": { "$date": "2026-03-10T03:00:00Z" } }
],
```

```
  "activo": true,
"fechaAlta": { "$date": "2026-10-07T16:22:00Z" }
}
// turnos
{
  "_id": 1,
  "mascotaId": 1,
  "fecha": { "$date": "2026-11-10T13:30:00Z" },
  "motivo": "Control anual",
  "estado": "pendiente",
  "activo": true,
  "fechaAlta": { "$date": "2026-10-31T20:00:00Z" }
}
```


Las fechas se guardan como tipo fecha (Date de BSON) y no como texto, lo que permite ordenarlas y compararlas correctamente. Se ingresan como AAAA-MM-DD HH:mm y el programa las convierte antes de guardarlas.


```
// contadores (colección auxiliar)
{ "_id": "clientes", "seq": 3 }
```


Justificación de Embedding y Linking

En MongoDB hay dos formas de relacionar datos. Embedding consiste en guardar los datos relacionados dentro del mismo documento: permite leer todo con una sola consulta, pero el documento crece. Linking, en cambio, consiste en guardar en un documento el identificador de otro: evita duplicar datos y mantiene los documentos chicos, pero requiere una consulta adicional para unirlos.
El criterio que aplicamos fue: se incrusta lo que es pequeño, acotado y se lee siempre junto al padre; se referencia lo que puede crecer sin límite o tiene vida propia.

Decisión	Técnica	Justificación
Dirección dentro de clientes	Embedding	Es un dato pequeño que siempre se consulta junto con el cliente y no tiene sentido por sí solo
vacunas dentro de mascotas	Embedding	Son pocas, pertenecen a una única mascota y se muestran siempre con ella
mascotas.clienteId	Linking	Un cliente puede tener muchas mascotas; al referenciarlo, si el cliente cambia un dato se modifica en un solo lugar
turnos.mascotaId	Linking	Los turnos crecen indefinidamente con el tiempo; incrustarlos haría crecer sin límite el documento de la mascota

Baja lógica

Ningún documento se elimina físicamente. Todas las colecciones incluyen el campo activo (booleano). Dar de baja un registro significa cambiar activo a false y guardar la fecha en fechaBaja; el documento permanece en la base.
Esto se aplica de la siguiente manera en el código:

- **Lectura:** solo devuelve los documentos con activo: true.
- **Actualización y baja:** solo operan sobre documentos activos.
- **Alta de mascotas y turnos:** se valida que el cliente o la mascota referenciados existan y estén activos.

Las ventajas son conservar el historial, poder auditar o recuperar un registro dado de baja por error, y evitar referencias rotas. Además, el backup (sección 6) incluye también los registros dados de baja.

3. Identificadores autoincrementales

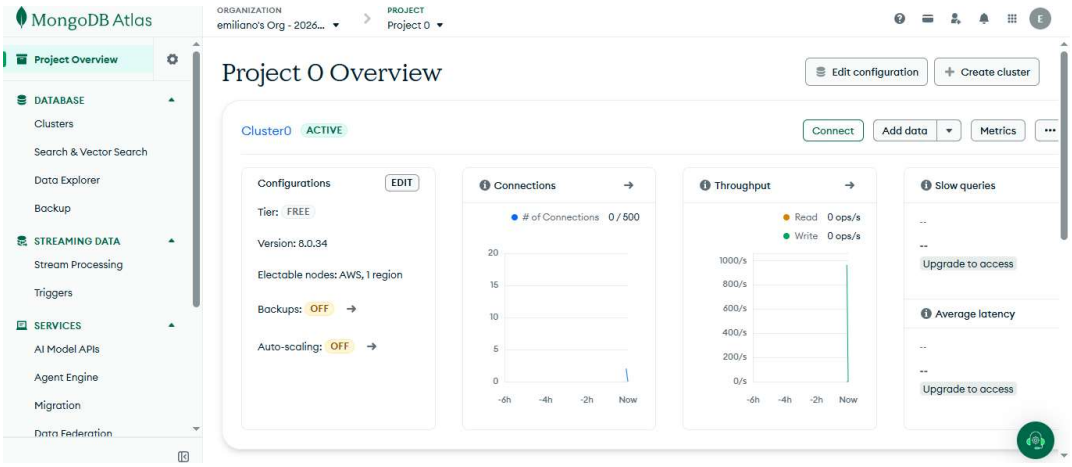
MongoDB asigna por defecto un ObjectId como _id. Para que los identificadores sean cortos y fáciles de usar (1, 2, 3...), se implementó un contador propio: la colección auxiliar contadores guarda un documento por colección con el último número usado. Cada vez que se crea un registro, el programa incrementa ese contador con el operador $inc, que es atómico: aunque dos usuarios inserten a la vez, nunca se repite un número.

La desventaja es que depende de una colección extra y que no escala tan bien en sistemas distribuidos, motivo por el cual MongoDB usa ObjectId por defecto. Para el alcance de este trabajo se consideró una solución adecuada.

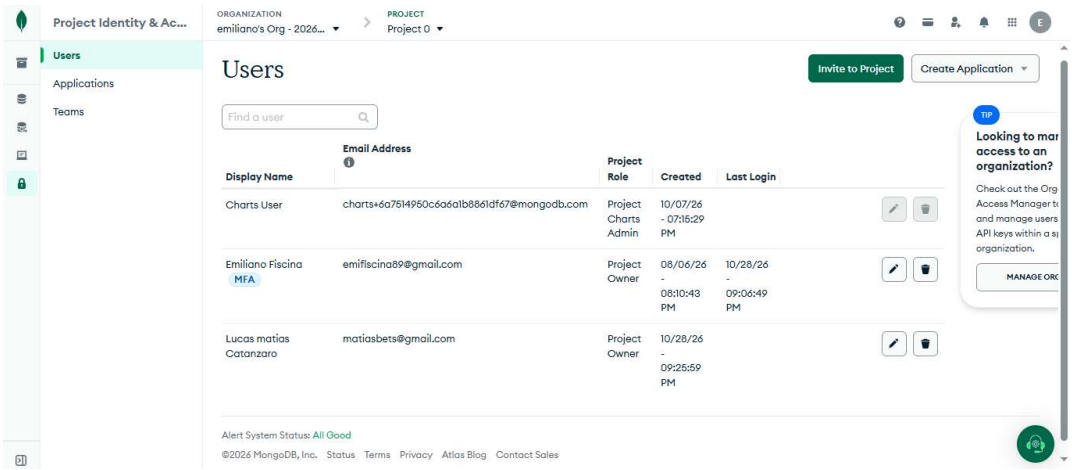
4. Persistencia en la nube: MongoDB Atlas

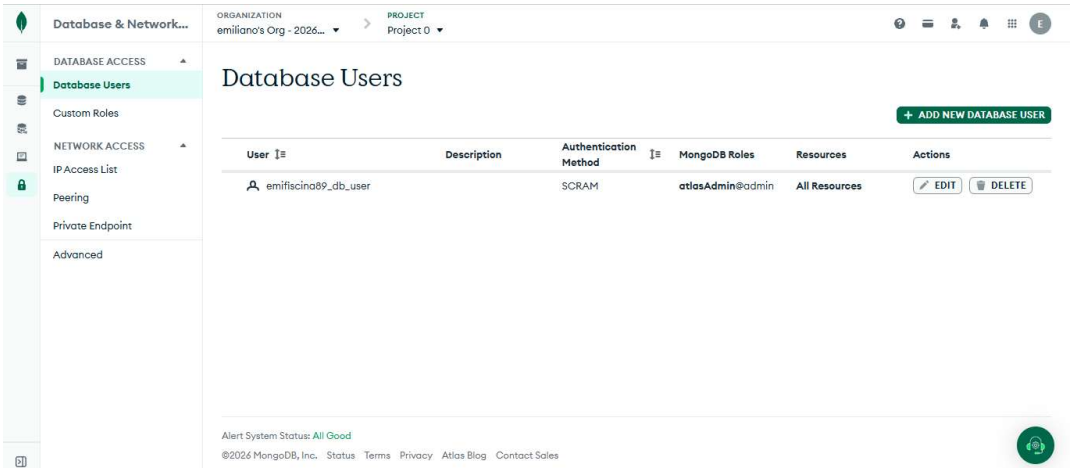
Se usó MongoDB Atlas, el servicio en la nube de MongoDB, con un cluster gratuito M0 (Cluster0, alojado en AWS). La configuración incluyó:

- Database Access: se creó un usuario de base de datos (emifiscina89_db_user) con rol de lectura y escritura, que es el que usa la aplicación para autenticarse.
- Network Access: se habilitó el acceso desde cualquier IP (0.0.0.0/0) para que el cluster pueda ser utilizado desde la aplicación y desde la corrección.
- Connection string: dirección de conexión en formato mongodb+srv://, que contiene el usuario, la contraseña y el cluster.

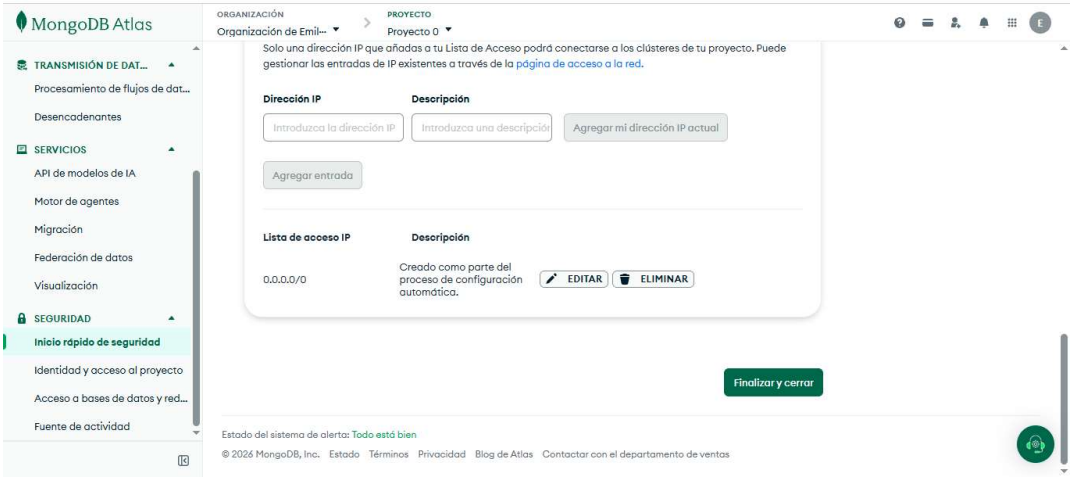


Cluster0 (plan gratuito M0) activo en MongoDB Atlas.

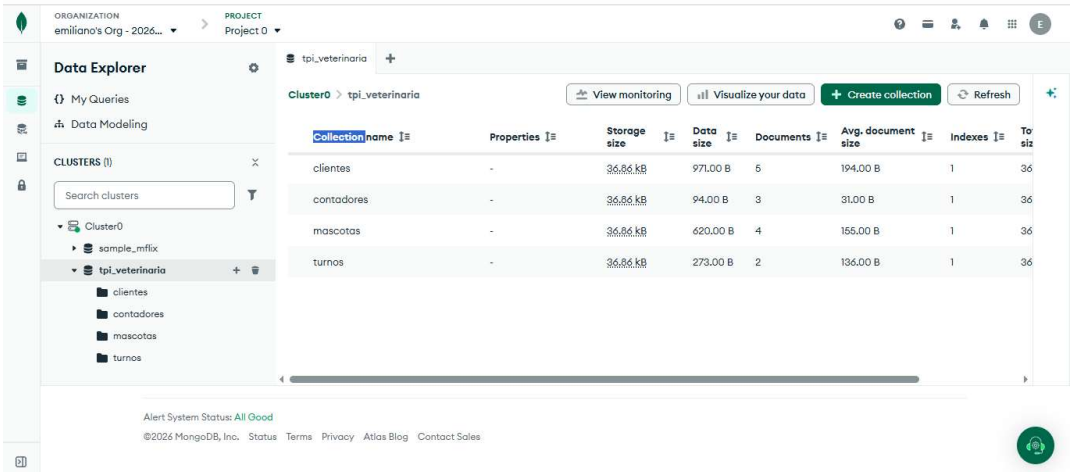




Database Access

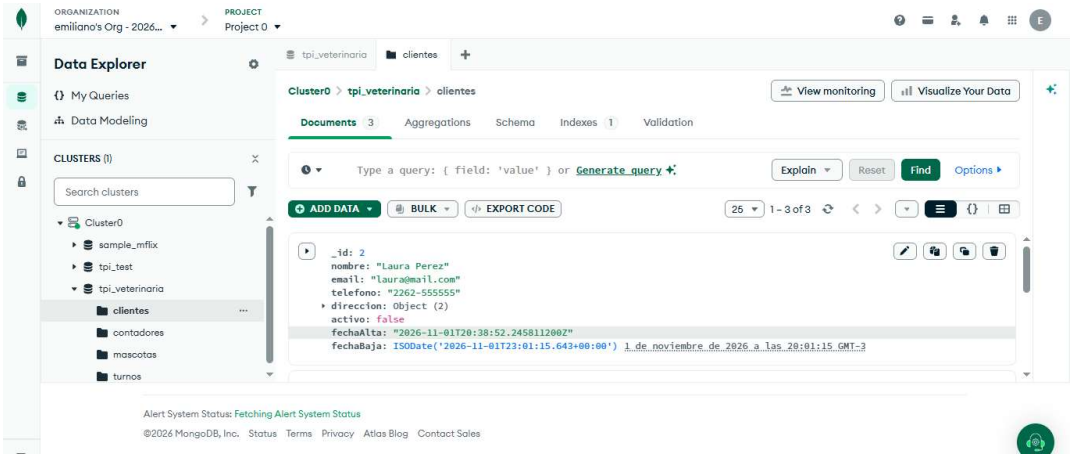


Network Access – IP Access List con 0.0.0.0/0

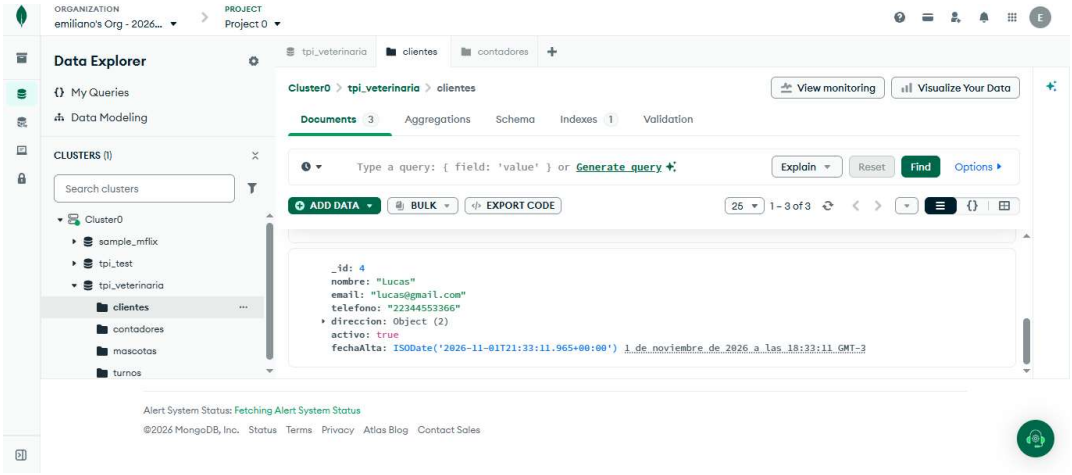


Browse Collections – colecciones clientes, mascotas, turnos y contadores con documentos

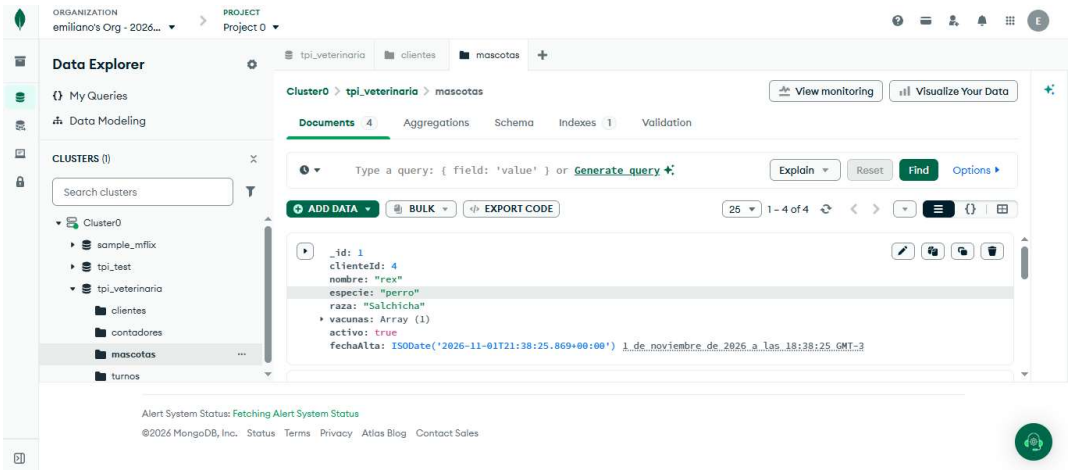
Colecciones clientes datos anterior al update de forma de fecha en texto



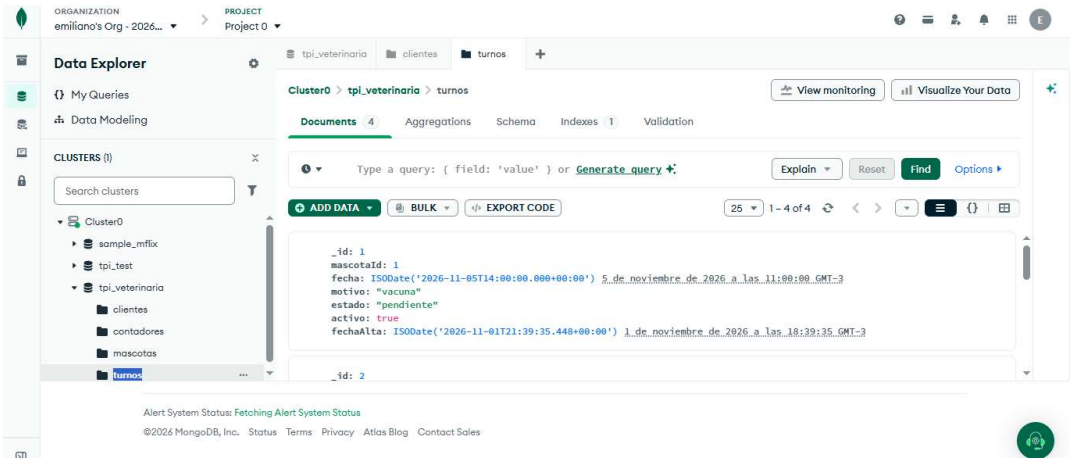
Se modifica para ver la fecha y que no sea guardada como texto



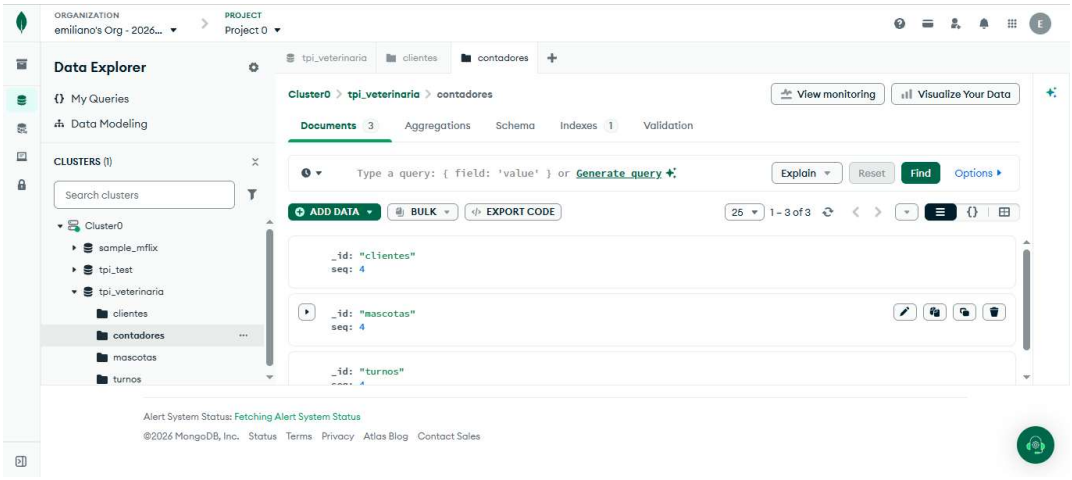
Mascotas



Turnos



Contadores



5. Aplicación y operaciones CRUD

Tecnología: La aplicación está desarrollada en Java 17, con Maven para gestionar dependencias y el driver nativo mongodb-driver-sync de MongoDB. Es una aplicación de consola con un menú que permite operar sobre las tres colecciones. Las credenciales no están escritas en el código: se leen desde un archivo.env.
Comunicación Cliente-Servidor: la aplicación Java actúa como cliente y MongoDB Atlas como servidor. El cliente se conecta mediante el connection string, envía las operaciones por internet y el servidor las ejecuta y devuelve el resultado. Al iniciar, el programa verifica la conexión con un comando ping.

Operaciones

Operación	Método del driver	Qué hace
CREATE	insertOne	Inserta un documento nuevo con ID autoincremental, activo: true y fecha de alta
READ	find con filtro activo: true	Lista solo los registros activos
UPDATE	updateOne con $set	Modifica un campo de una lista de campos permitidos, convirtiendo el valor al tipo correcto (texto, estado, fecha o referencia validada)
DELETE	updateOne con $set	Cambia activo a false y registra fechaBaja

Además, se agregó un $lookup (equivalente a un JOIN) para listar las mascotas junto con los datos de su cliente y mostrar así el funcionamiento del linking.

Código de la aplicación

A continuación se muestran los fragmentos principales de App.java (clase tpi.App). El proyecto completo se entrega en el ZIP.

Conexión con MongoDB Atlas

```
Map<String, String> env = leerEnv();  // lee MONGO_URI y DB_NAME desde .env
try (MongoClient client = MongoClients.create(env.get("MONGO_URI"))) {
  db = client.getDatabase(env.get("DB_NAME"));
  db.runCommand(new Document("ping", 1));  // verifica la conexion
  System.out.println("Conectado a MongoDB Atlas");
  // ... menu principal ...
}
```

ID autoincremental (colección auxiliar contadores)

```
static int siguienteId(String col) {
  Document r = db.getCollection("contadores").findOneAndUpdate(
      Filters.eq("_id", col),
      Updates.inc("seq", 1),   // $inc es atomico
      new FindOneAndUpdateOptions().upsert(true)
          .returnDocument(ReturnDocument.AFTER));
  return r.getInteger("seq");
}
```

CREATE – clientes (embedding de la dirección)

```
doc.append("nombre", pedir("Nombre"))
  .append("email", pedir("Email"))
  .append("telefono", pedir("Telefono"));
doc.append("direccion", new Document("calle", pedir("Calle"))
      .append("ciudad", pedir("Ciudad")));   // EMBEDDING
...
Document nuevo = new Document("_id", siguienteId(col));
nuevo.putAll(doc);
nuevo.append("activo", true).append("fechaAlta", new Date());
db.getCollection(col).insertOne(nuevo);
```

CREATE – mascotas (linking con el cliente y embedding de vacunas)

```
int clienteId = pedirInt("ID del cliente duenio");
if (!existeActivo("clientes", clienteId)) {   // LINKING validado
  System.out.println("El cliente no existe o esta dado de baja");
  return;
}
doc.append("clienteId", clienteId)
  .append("nombre", pedir("Nombre"))
  .append("especie", pedir("Especie"))
  .append("raza", pedir("Raza"));
```

```
    // vacunas: lista de documentos incrustados
    doc.append("vacunas", vacunas);
```

READ – solo registros activos

```
    for (Document d : db.getCollection(col).find(Filters.eq("activo", true))) {
        System.out.println("ID " + d.get("_id") + " -> " + d.toJson());
    }
```

UPDATE – lista blanca de campos editables y conversión de tipos

```
    static final Map<String, Map<String, String>> EDITABLES = Map.of(
        "clientes", Map.of("nombre", "texto", "email", "texto", "telefono", "texto",
                "direccion.calle", "texto", "direccion.ciudad", "texto"),
        "mascotas", Map.of("nombre", "texto", "especie", "texto", "raza", "texto",
                "clienteId", "ref:clientes"),
        "turnos", Map.of("fecha", "fechahora", "motivo", "texto", "estado", "estado",
                "mascotaId", "ref:mascotas"));

    static void actualizar(String col) {
        int id = pedirInt("ID del documento");
        Map<String, String> campos = EDITABLES.get(col);
        System.out.println("Campos editables: " + String.join(", ", new
    TreeSet<>(campos.keySet())));
        String campo = pedir("Campo a modificar");
        String tipo = campos.get(campo);
        if (tipo == null) {
            System.out.println("Campo no editable (activo, fechaAlta, fechaBaja y _id estan
    protegidos)");
            return;
        }
        Object valor = convertir(tipo, pedir("Nuevo valor"));
        var r = db.getCollection(col).updateOne(
                Filters.and(Filters.eq("_id", id), Filters.eq("activo", true)),
                Updates.set(campo, valor));
        System.out.println(r.getMatchedCount() > 0 ? "Actualizado"
                : "No se encontro un documento activo con ese ID");
    }

    static Object convertir(String tipo, String texto) {
        if (texto.isBlank()) throw new IllegalArgumentException("el valor no puede estar
    vacio");
        if (tipo.equals("texto")) return texto;
        if (tipo.equals("estado")) {
            String e = texto.toLowerCase();
```

```
        if (!ESTADOS.contains(e)) throw new IllegalArgumentException("estado
    invalido");
        return e;
      }
      if (tipo.equals("fechahora")) return parseFechaHora(texto);
      if (tipo.startsWith("ref:")) {
        String colRef = tipo.substring(4);
        int refId = Integer.parseInt(texto.trim());
        if (!existeActivo(colRef, refId))
          throw new IllegalArgumentException("no existe un registro activo en " +
    colRef);
        return refId;
      }
      throw new IllegalArgumentException("tipo de campo desconocido");
    }
```

La actualización no acepta cualquier campo: cada colección tiene una lista blanca de campos editables, cada uno con su tipo. Los campos de control (_id, activo, fechaAlta y fechaBaja) están protegidos, de modo que activo solo cambia mediante la baja lógica. Antes de guardar, el valor se convierte al tipo del campo: los textos se guardan como texto; el estado solo admite pendiente, atendido o cancelado; las fechas se convierten a tipo fecha; y las referencias (clienteId, mascotaId) se guardan como número y se comprueba que el registro referenciado exista y esté activo.

DELETE – baja lógica (no se elimina el documento)

```
    var r = db.getCollection(col).updateOne(
        Filters.and(Filters.eq("_id", id), Filters.eq("activo", true)),
        Updates.combine(Updates.set("activo", false),
                Updates.set("fechaBaja", new Date())));
```

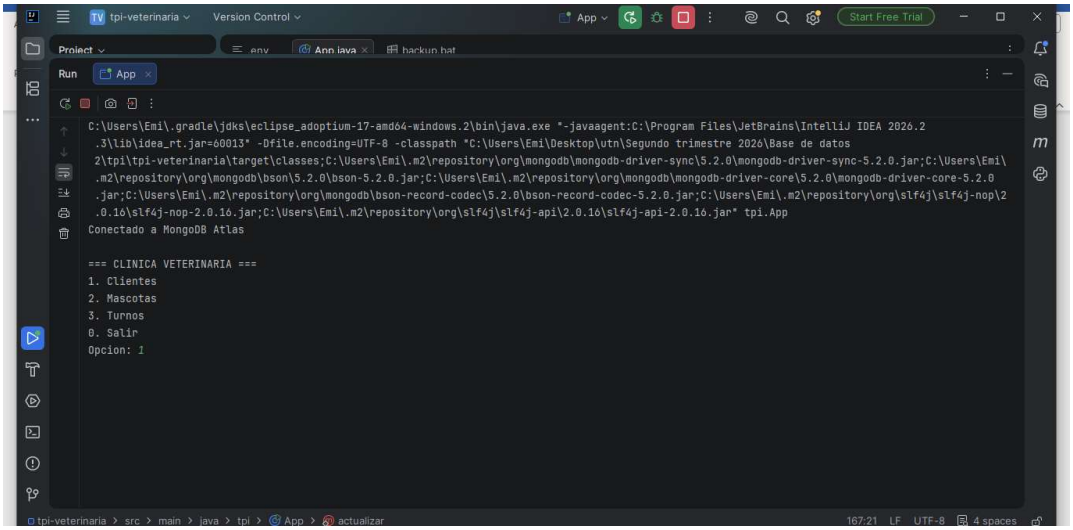
Validación de referencias y $lookup

```
    static boolean existeActivo(String col, int id) {
      return db.getCollection(col).countDocuments(Filters.and(
          Filters.eq("_id", id), Filters.eq("activo", true))) > 0;
    }

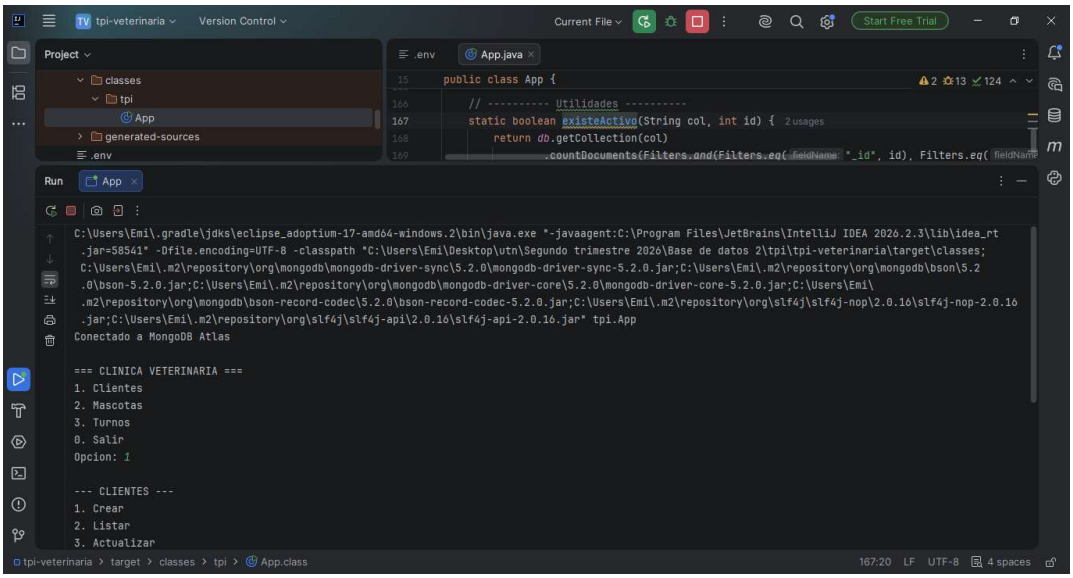
    db.getCollection("mascotas").aggregate(List.of(
        Aggregates.match(Filters.eq("activo", true)),
        Aggregates.lookup("clientes", "clienteId", "_id", "cliente")));
```

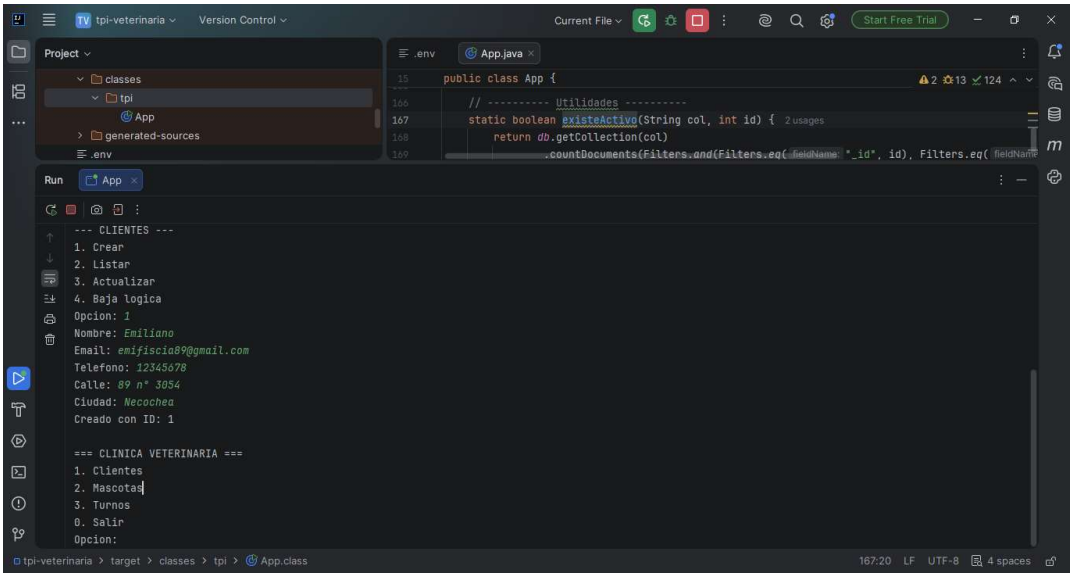
Pruebas realizadas

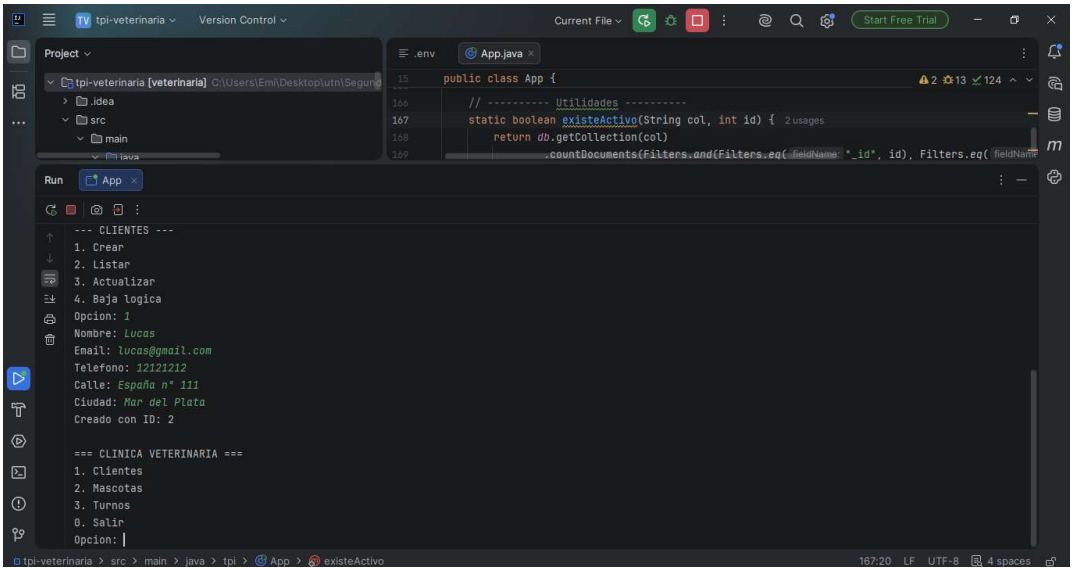
Conexión con base de datos y menú: el programa se conecta a Atlas y muestra el menú principal.

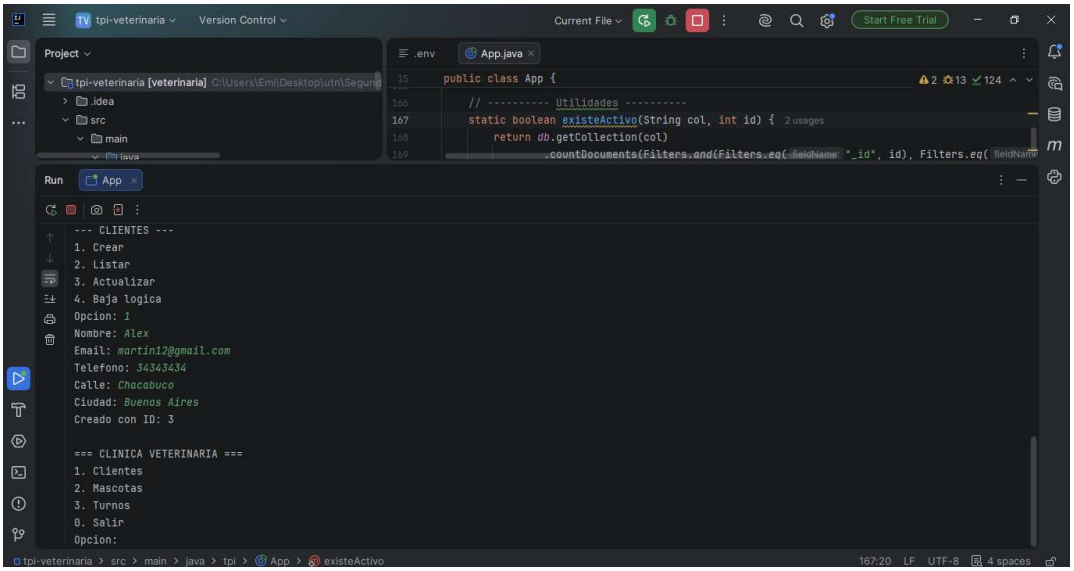


Creación de cliente: se carga un cliente con su dirección incrustada y el sistema devuelve el ID asignado.

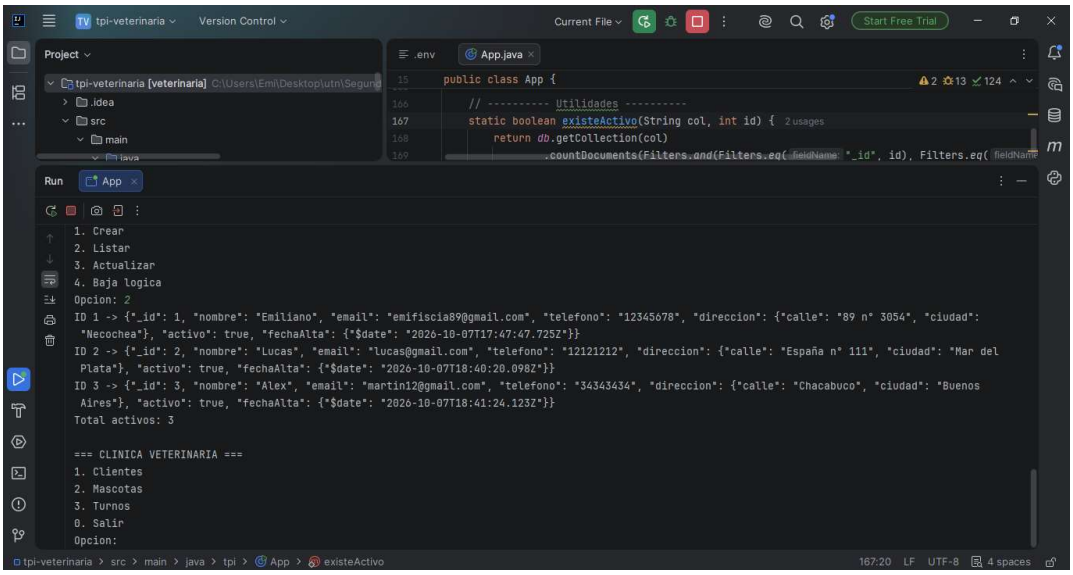



tpi-veterinaria
Version Control
Current File
Start Free Trial
Project
classes
tpi
App
generated-sources
.env
App.java
public class App {
// ---------- Utilidades ----------
static boolean existeActivo(String col, int id) { 2 usages
return db.getCollection(col)
.countDocuments(Filters.and(Filters.eq(fieldName "_id", id), Filters.eq(fieldName
Run
App
--- CLIENTES ---
1. Crear
2. Listar
3. Actualizar
4. Baja logica
Opcion: 1
Nombre: Emiliano
Email: emifiscia89@gmail.com
Telefono: 12345678
Calle: 89 n° 3054
Ciudad: Necochea
Creado con ID: 1
=== CLINICA VETERINARIA ===
1. Clientes
2. Mascotas
3. Turnos
0. Salir
Opcion:
tpi-veterinaria > target > classes > tpi > App.class
167:20 LF UTF-8 4 spaces

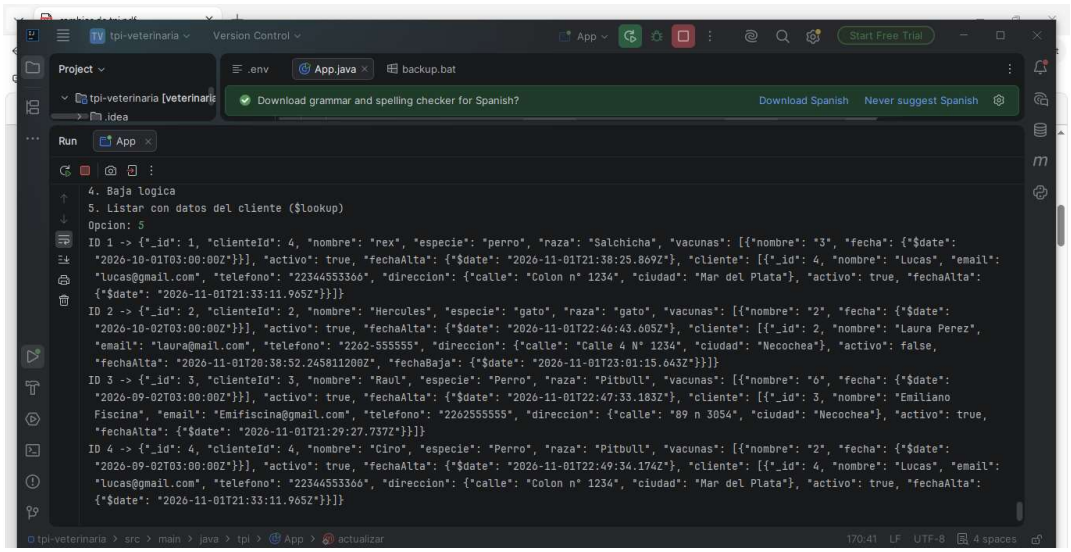

tpi-veterinaria
Version Control
Current File
Start Free Trial
Project
tpi-veterinaria [veterinaria] C:\Users\Emi\Desktop\utn\Segund
.idea
src
main
.env
App.java
public class App {
// ---------- Utilidades ----------
static boolean existeActivo(String col, int id) { 2 usages
return db.getCollection(col)
.countDocuments(Filters.and(Filters.eq(fieldName "_id", id), Filters.eq(fieldName
Run
App
--- CLIENTES ---
1. Crear
2. Listar
3. Actualizar
4. Baja logica
Opcion: 1
Nombre: Lucas
Email: lucas@gmail.com
Telefono: 12121212
Calle: España n° 111
Ciudad: Mar del Plata
Creado con ID: 2
=== CLINICA VETERINARIA ===
1. Clientes
2. Mascotas
3. Turnos
0. Salir
Opcion:
tpi-veterinaria > src > main > java > tpi > App > existeActivo
167:20 LF UTF-8 4 spaces


tpi-veterinaria
Version Control
Current File
Start Free Trial
Project
tpi-veterinaria [veterinaria] C:\Users\Emi\Desktop\utn\Segund
.idea
src
main
.env
App.java
public class App {
// ---------- Utilidades ----------
static boolean existeActivo(String col, int id) { 2 usages
return db.getCollection(col)
.countDocuments(Filters.and(Filters.eq(fieldName "_id", id), Filters.eq(fieldName
Run
App
--- CLIENTES ---
1. Crear
2. Listar
3. Actualizar
4. Baja logica
Opcion: 1
Nombre: Alex
Email: martin12@gmail.com
Telefono: 34343434
Calle: Chacabuco
Ciudad: Buenos Aires
Creado con ID: 3
=== CLINICA VETERINARIA ===
1. Clientes
2. Mascotas
3. Turnos
0. Salir
Opcion:
tpi-veterinaria > src > main > java > tpi > App > existeActivo
167:20 LF UTF-8 4 spaces

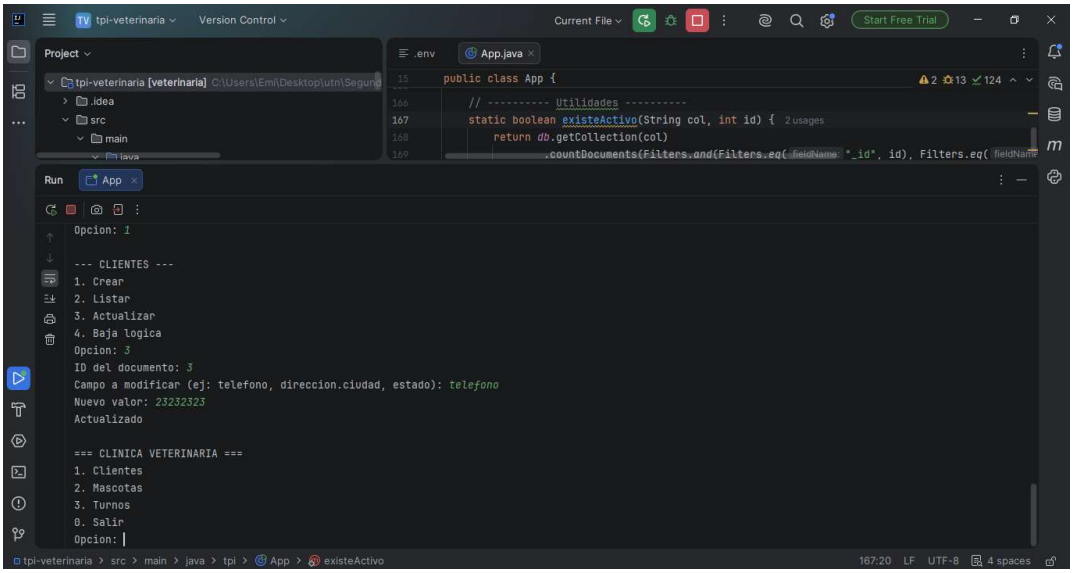
Listado de clientes: se muestran los clientes activos.



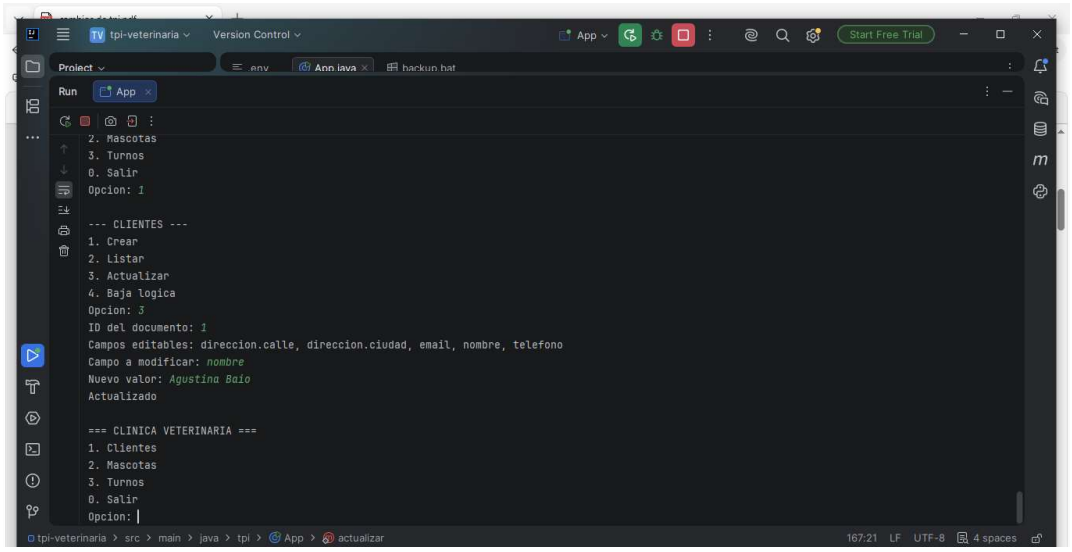
Listar clientes y mascotas juntos



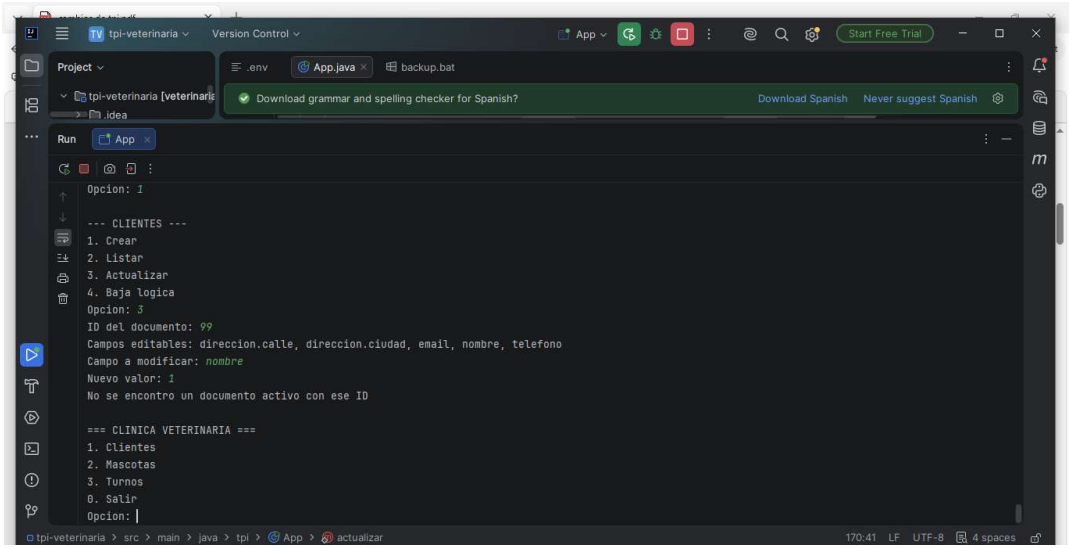
Actualización de datos: se modifica un campo (por ejemplo, el teléfono) de un cliente.



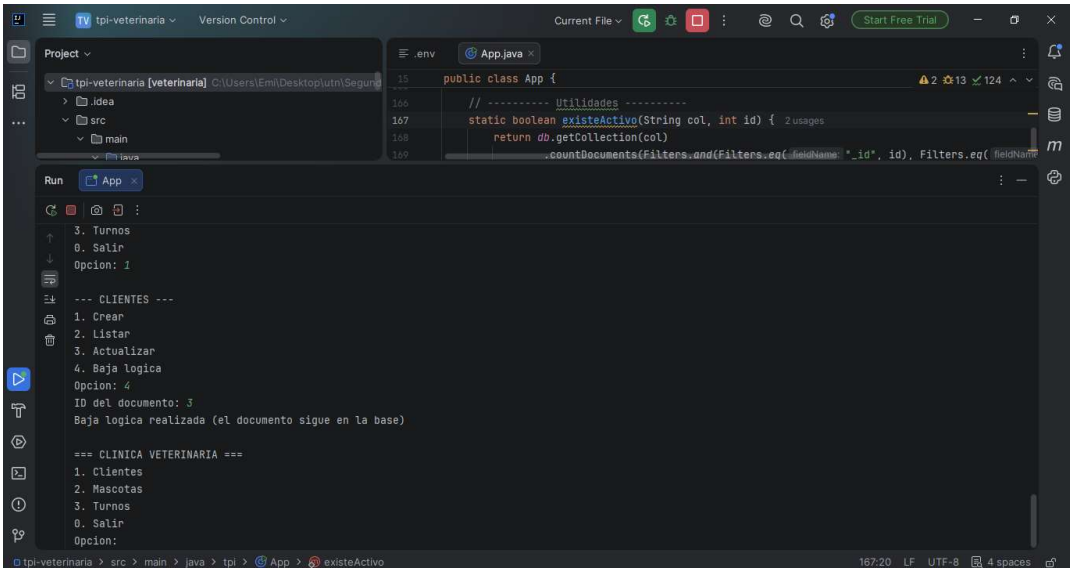
Captura nueva de campos editables en actualizacion 1/11/2026



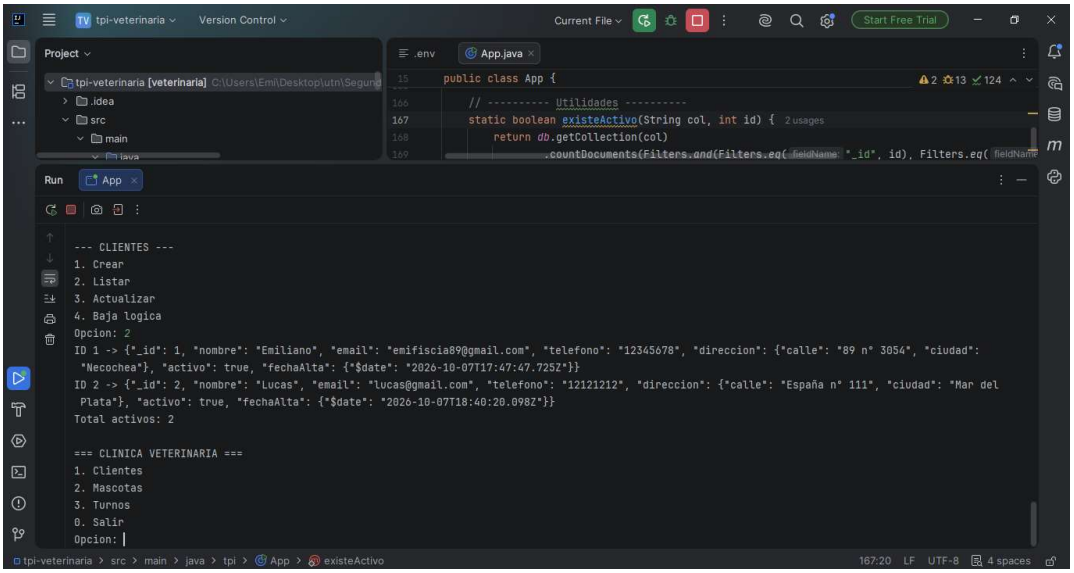
Captura de actualizar datos de clientes por uno inexistente



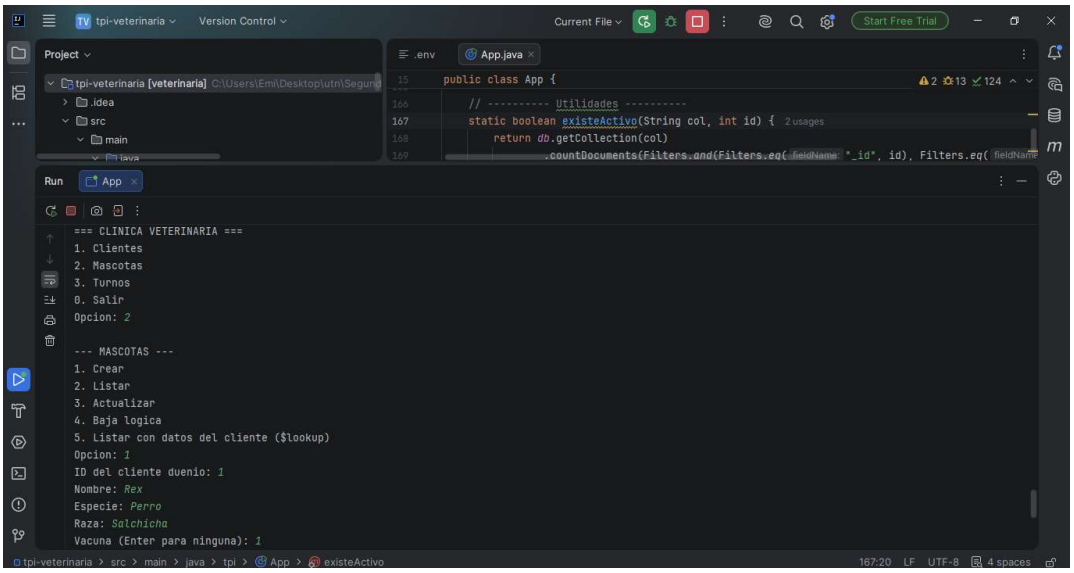
Baja de un cliente: se realiza la baja lógica; el documento no se elimina.

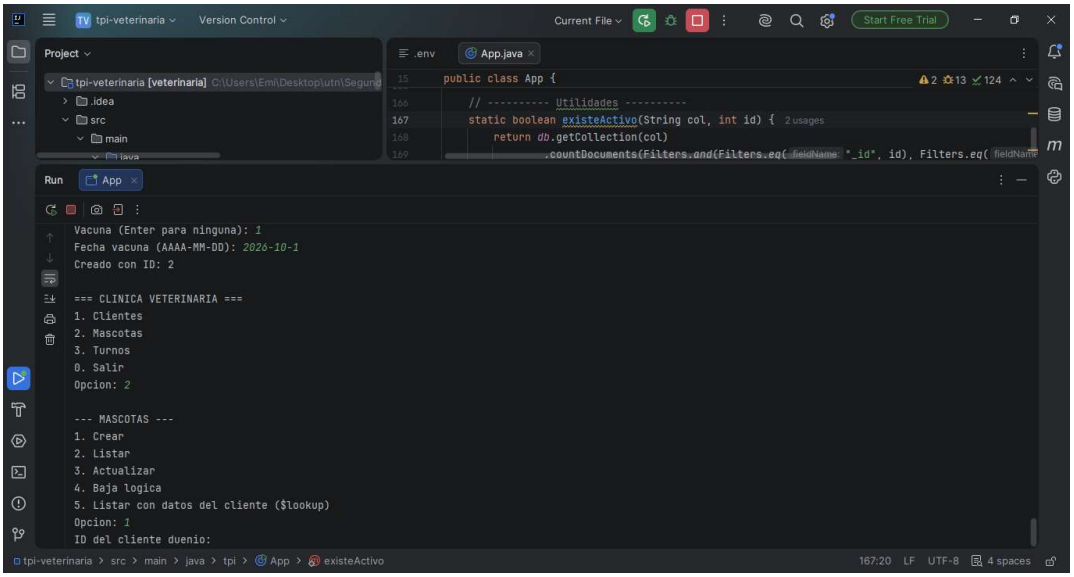


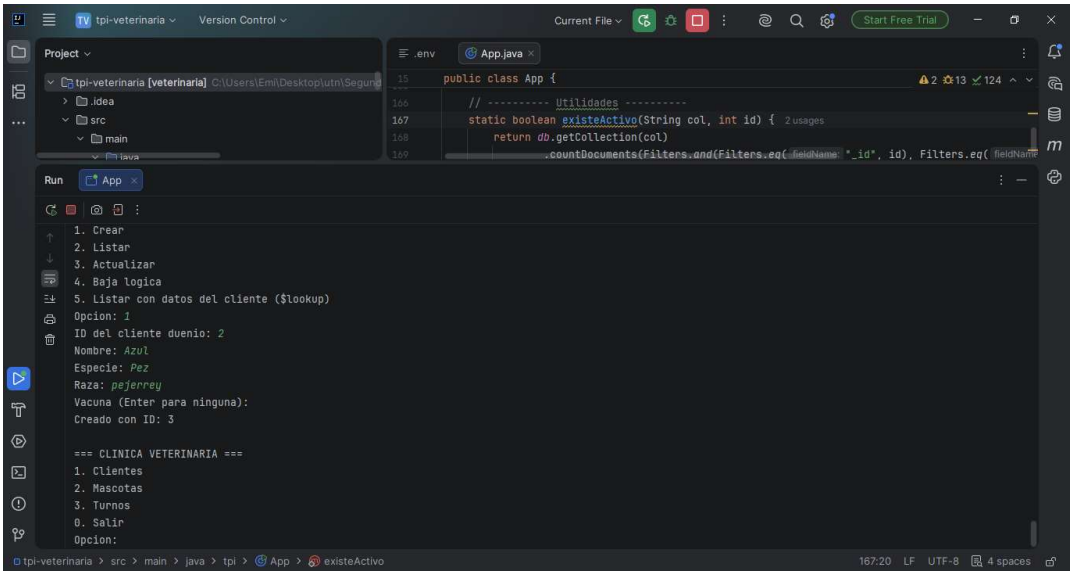
Listado con el cliente borrado: el cliente dado de baja ya no aparece en el listado, aunque sigue existiendo en la base con activo: false.

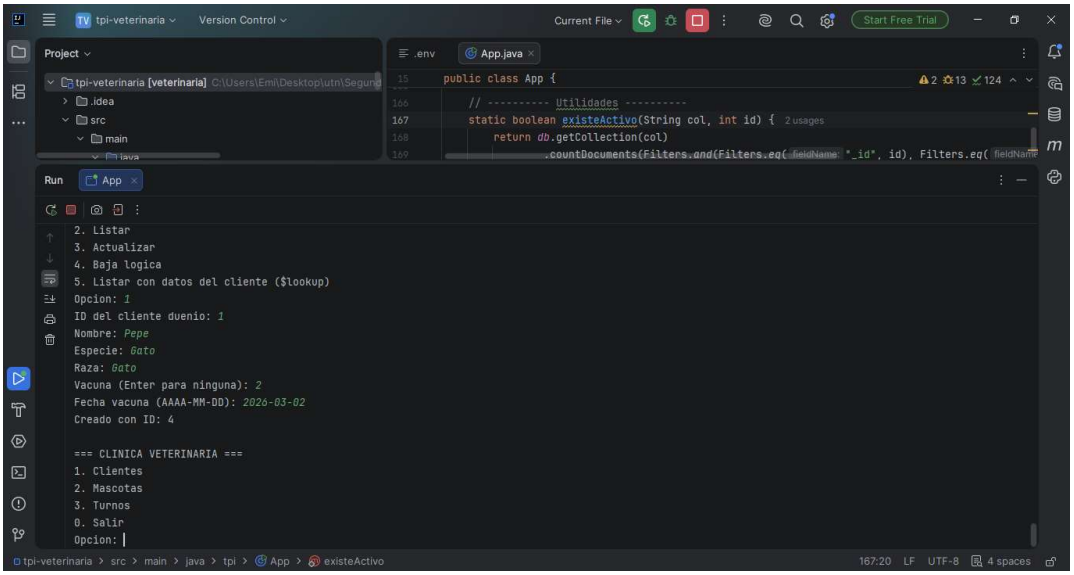


Creación de mascota: se vincula a un cliente existente y activo mediante clienteId.



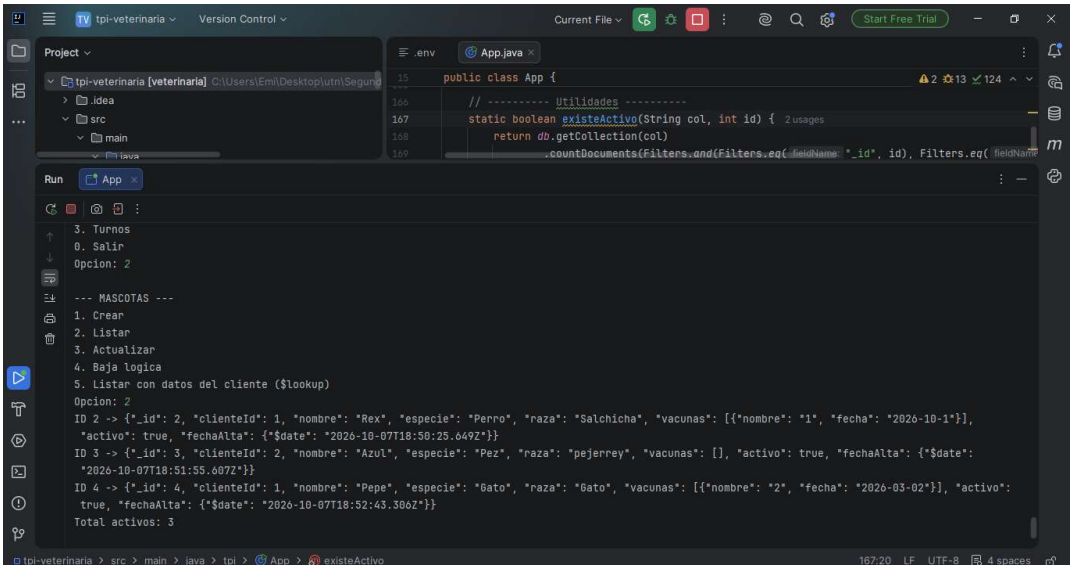

tpi-veterinaria
Version Control
Current File
Start Free Trial
Project
tpi-veterinaria [veterinaria] C:\Users\Emi\Desktop\utn\Segund
.idea
src
main
.env
App.java
public class App {
// ---------- Utilidades ----------
static boolean existeActivo(String col, int id) { 2 usages
return db.getCollection(col)
.countDocuments(Filters.and(Filters.eq(fieldName "_id", id), Filters.eq(fieldName
Run
App
Vacuna (Enter para ninguna): 1
Fecha vacuna (AAAA-MM-DD): 2026-10-1
Creado con ID: 2
=== CLINICA VETERINARIA ===
1. Clientes
2. Mascotas
3. Turnos
0. Salir
Opcion: 2
--- MASCOTAS ---
1. Crear
2. Listar
3. Actualizar
4. Baja logica
5. Listar con datos del cliente ($lookup)
Opcion: 1
ID del cliente duenio:
tpi-veterinaria > src > main > java > tpi > App > existeActivo
167:20 LF UTF-8 4 spaces


tpi-veterinaria
Version Control
Current File
Start Free Trial
Project
tpi-veterinaria [veterinaria] C:\Users\Emi\Desktop\utn\Segund
.idea
src
main
.env
App.java
public class App {
// ---------- Utilidades ----------
static boolean existeActivo(String col, int id) { 2 usages
return db.getCollection(col)
.countDocuments(Filters.and(Filters.eq(fieldName "_id", id), Filters.eq(fieldName
Run
App
1. Crear
2. Listar
3. Actualizar
4. Baja logica
5. Listar con datos del cliente ($lookup)
Opcion: 1
ID del cliente duenio: 2
Nombre: Azul
Especie: Pez
Raza: pejerrey
Vacuna (Enter para ninguna):
Creado con ID: 3
=== CLINICA VETERINARIA ===
1. Clientes
2. Mascotas
3. Turnos
0. Salir
Opcion:
tpi-veterinaria > src > main > java > tpi > App > existeActivo
167:20 LF UTF-8 4 spaces

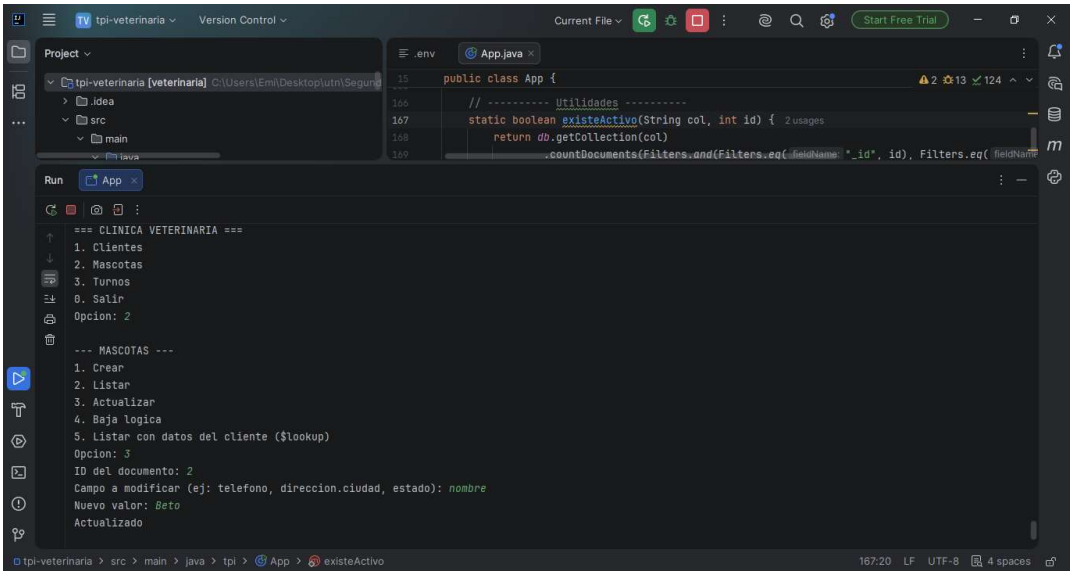


Listado, actualización y baja de mascotas: mismo ciclo que en clientes.

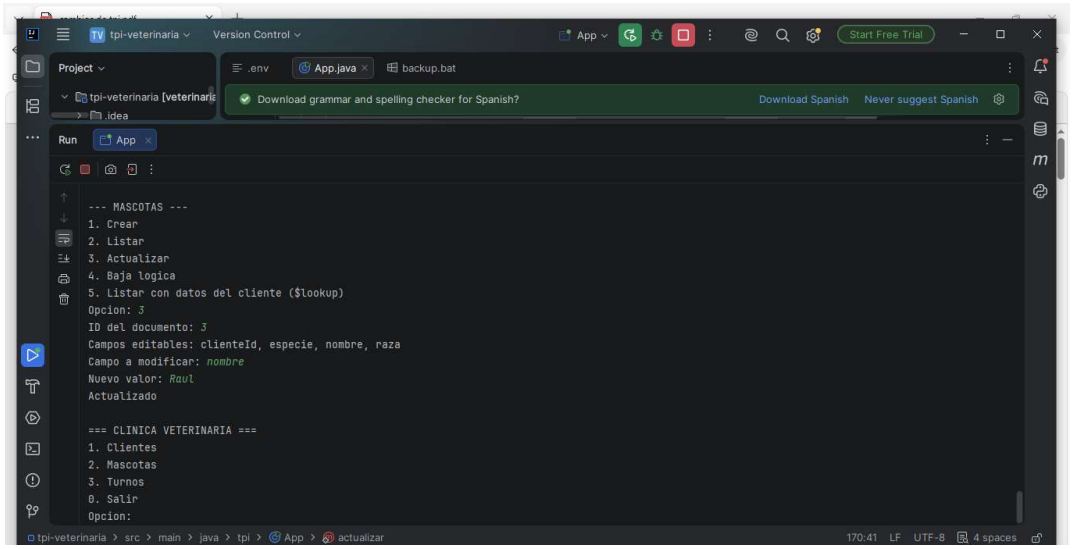
Listar mascotas



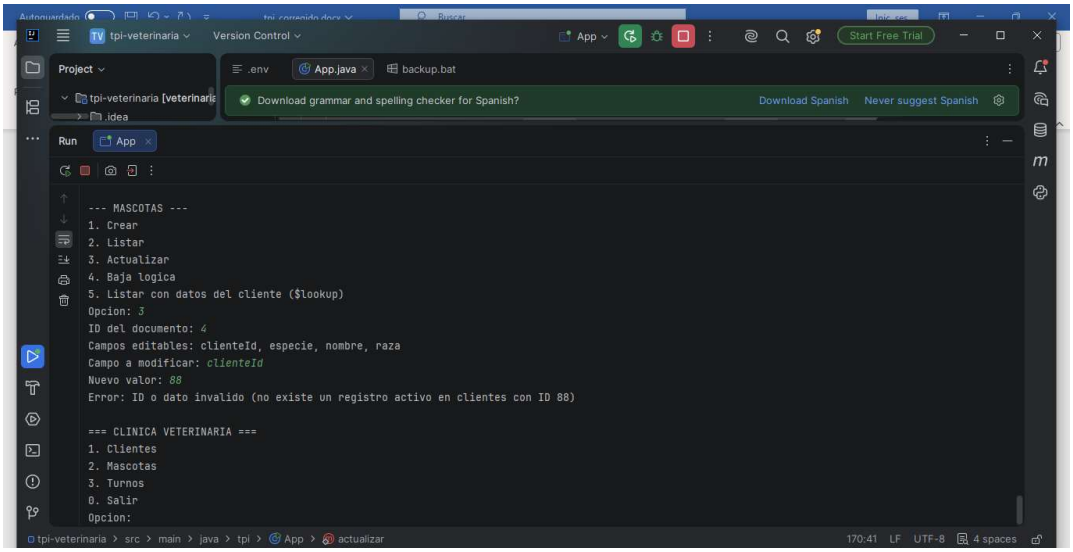
Actualizar mascotas



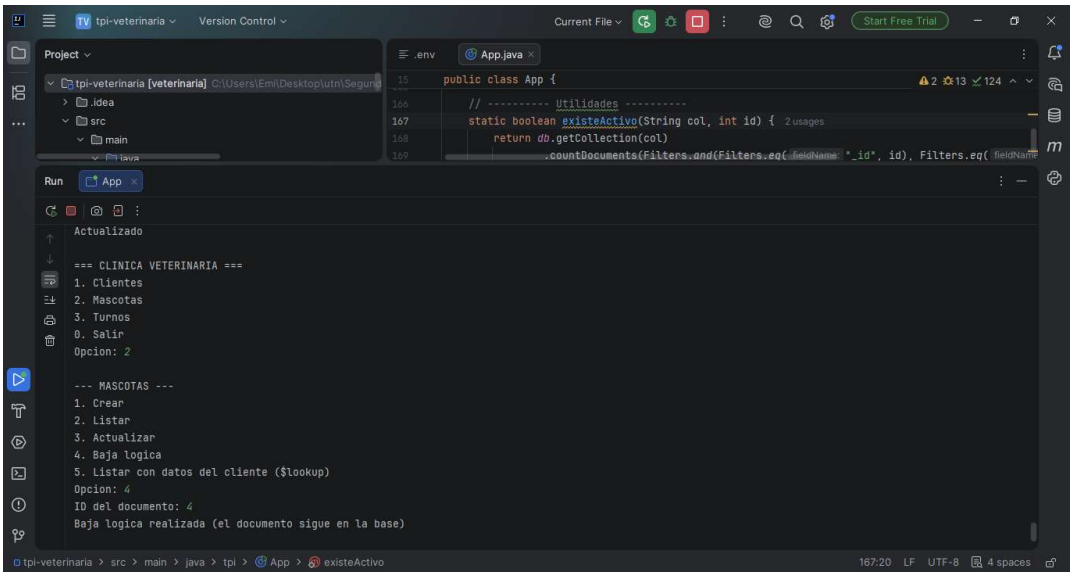
Cambio de update de mascota



Tratar de hacer update y tratas de modificar clienteId da error

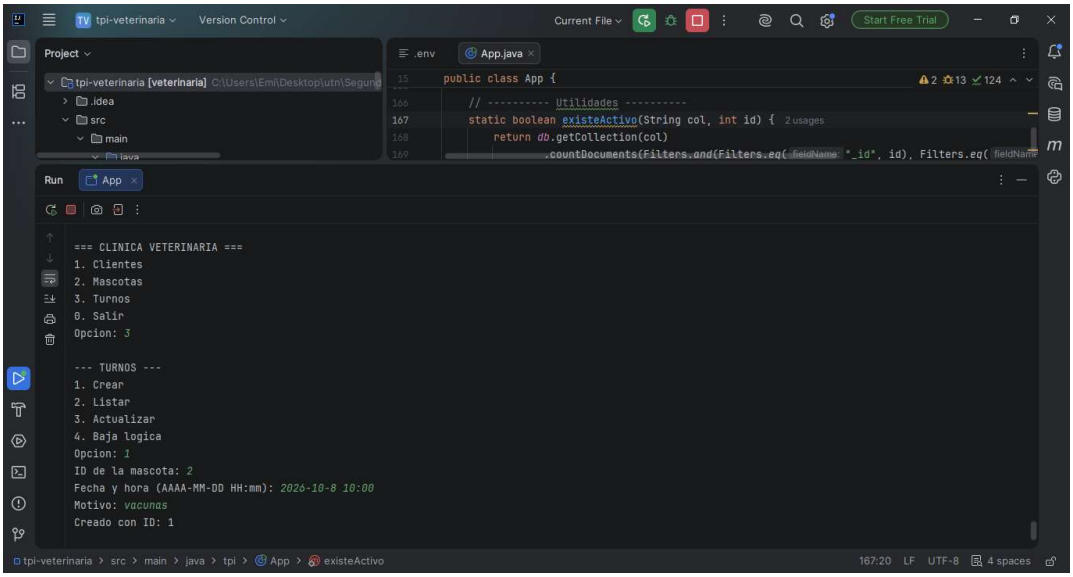


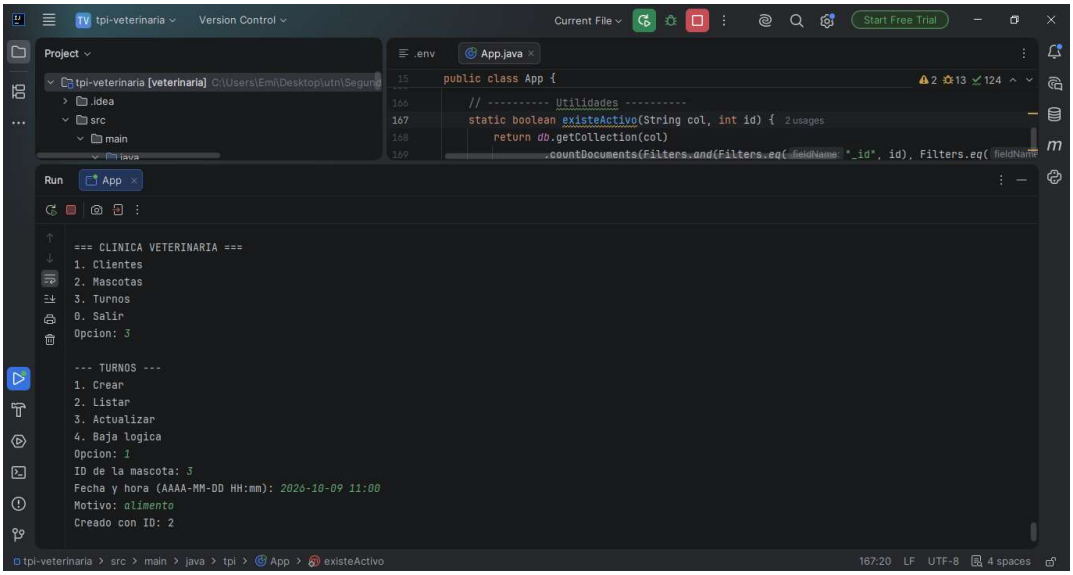
Baja de mascotas



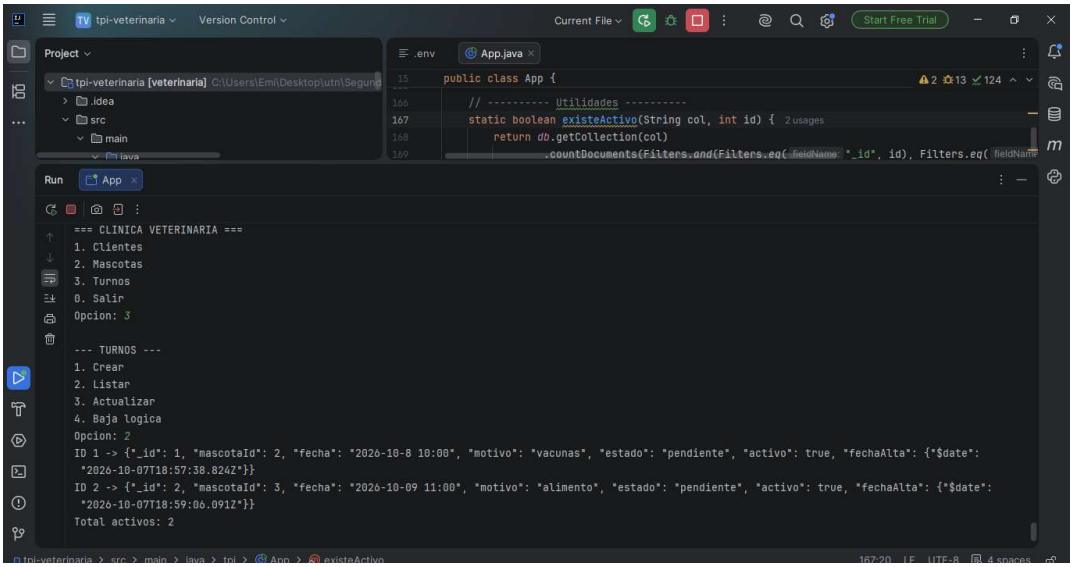
Creación, listado y baja de turnos: se vinculan a una mascota mediante mascotaId.

Creación de turno

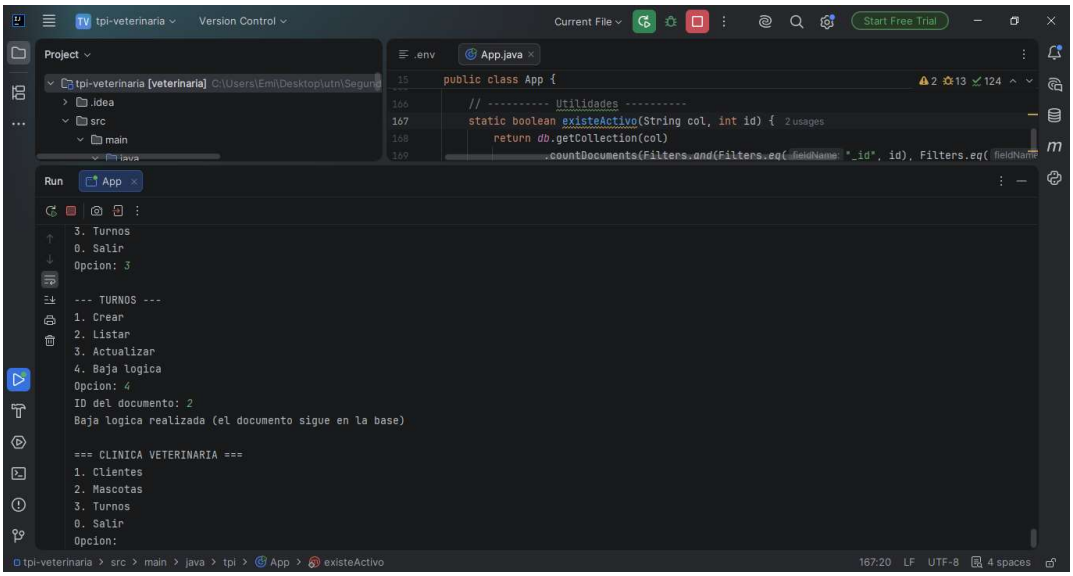




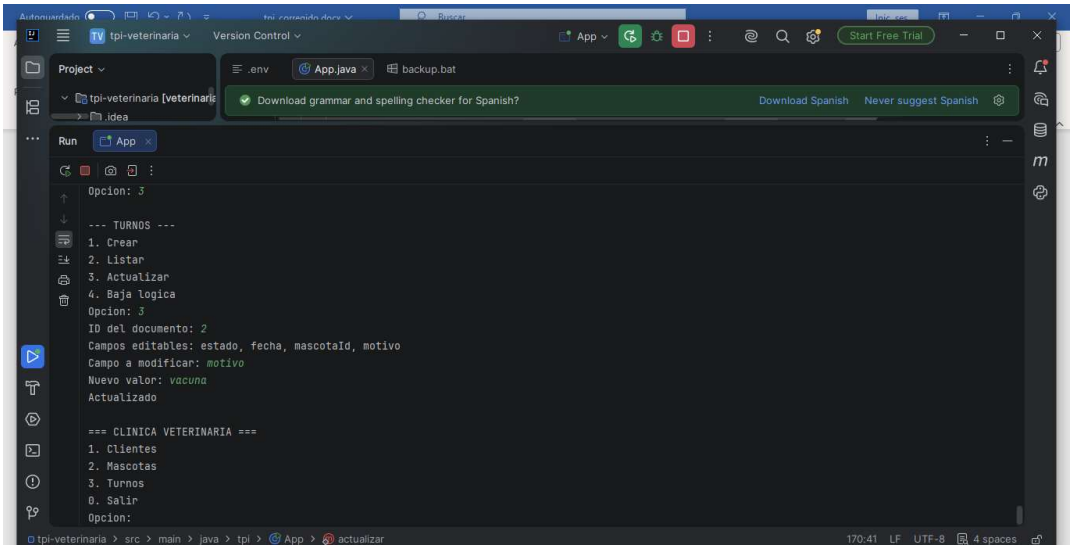
Listado de turnos



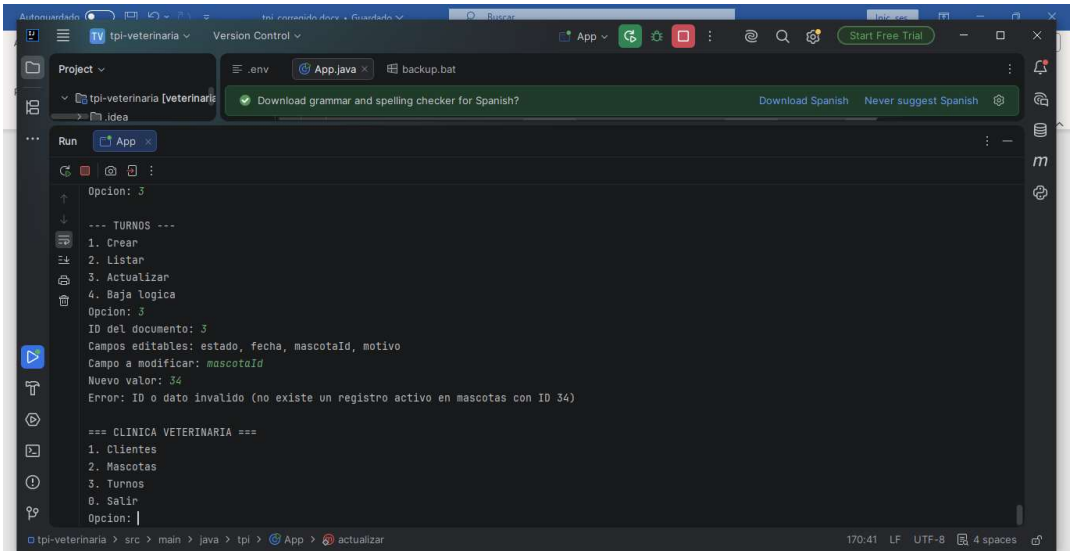
Baja de turno



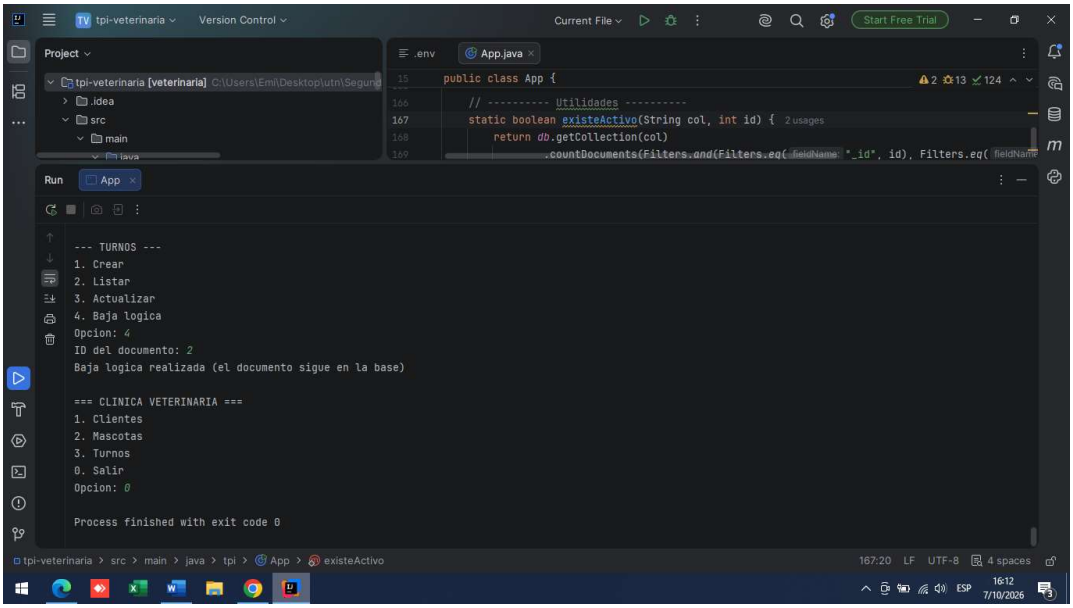
Actualización de turno cambiando el motivo



Tratar de modificar id del documento



Salir: cierre ordenado del programa y de la conexión.



6. Backup del trabajo

Se automatizó el respaldo con un script de Windows, backup.bat, que utiliza la herramienta mongodump de MongoDB Database Tools. El script realiza lo siguiente:

- Se posiciona en su propia carpeta, por lo que usa rutas relativas y funciona en cualquier equipo.
- Lee la URI de conexión desde el archivo.env.
- Obtiene la fecha y hora actuales.
- Crea la carpeta resguardos_tpi\<fecha-hora>.
- Ejecuta mongodump indicando con --db la base a respaldar (tomada de DB_NAME): se conecta remotamente a Atlas y descarga esa base dentro de la carpeta.
- Verifica el resultado: si mongodump falla, informa el error, elimina la carpeta vacía y no muestra el mensaje de éxito.

Script

```
@echo off

cd /d "%~dp0"

for /f "usebackq tokens=1,* delims==" %%a in (".env") do set %%a=%%b

for /f %%i in ('powershell -command "Get-Date -Format yyyy-MM-dd_HH-mm-ss"') do set FECHA=%%i

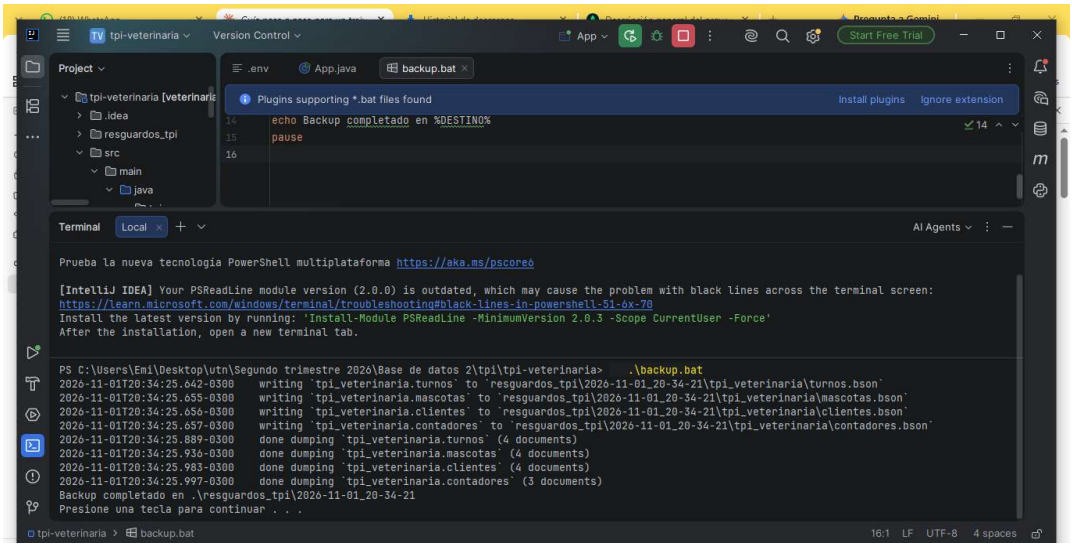
set DESTINO=.\resguardos_tpi\%FECHA%

mkdir "%DESTINO%"

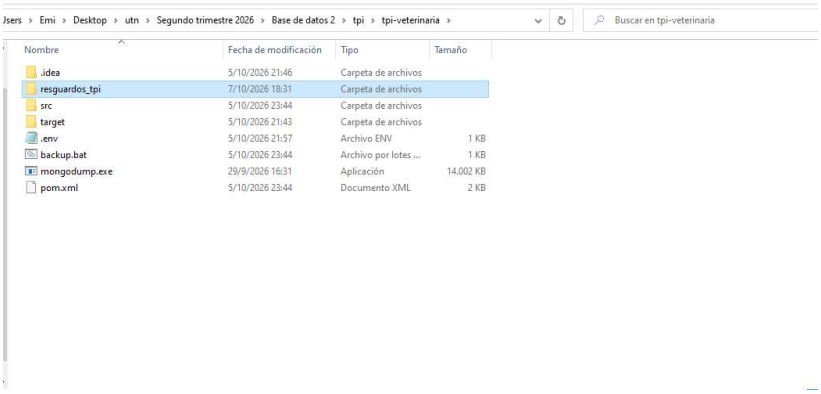
mongodump --uri="%MONGO_URI%" --db="%DB_NAME%" --out="%DESTINO%"
```

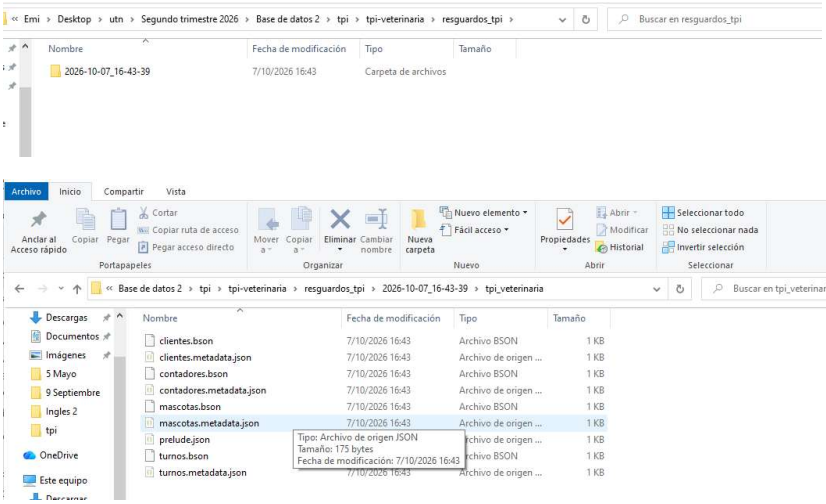
```
if errorlevel 1 (
    echo ERROR: el backup fallo. Revise el mensaje de arriba.
    rmdir "%DESTINO%" 2>nul
    pause
    exit /b 1
)
echo Backup completado en %DESTINO%
pause
```

En la ejecución realizada se respaldaron las cuatro colecciones: clientes (4 documentos), mascotas (4), turnos (4) y contadores (3), incluyendo los registros dados de baja. Se generó la carpeta resguardos_tpi\<fecha-hora>\tpi_veterinaria\ con un archivo .bson por colección. El proceso tardó pocos segundos.



Carpeta resultante del backup



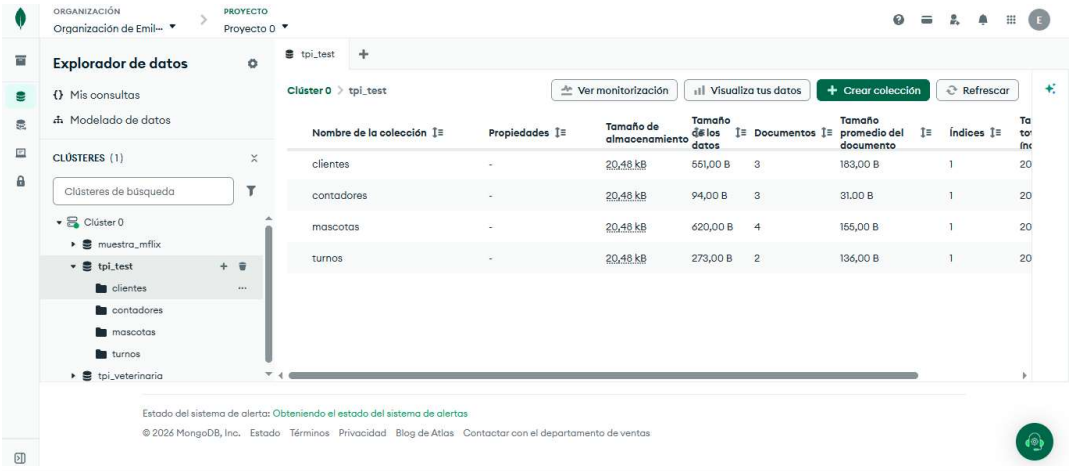


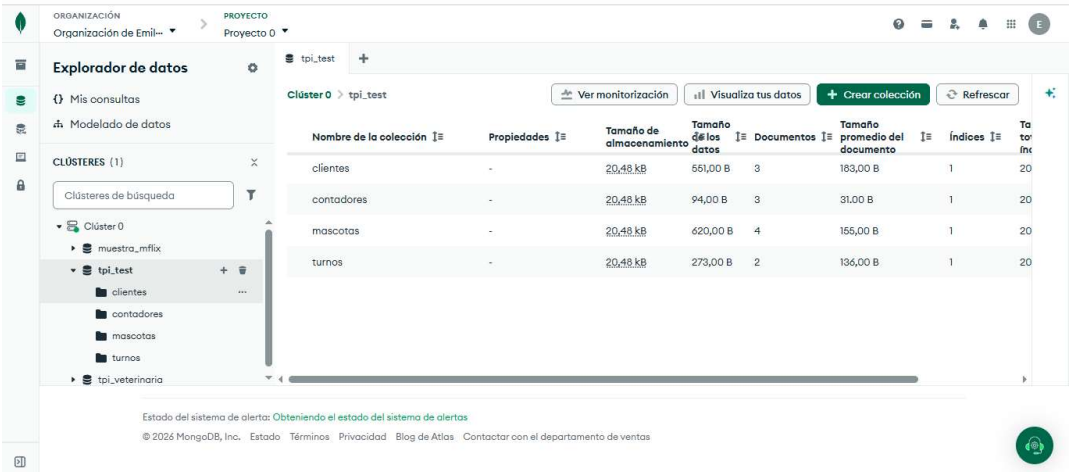
Restauración

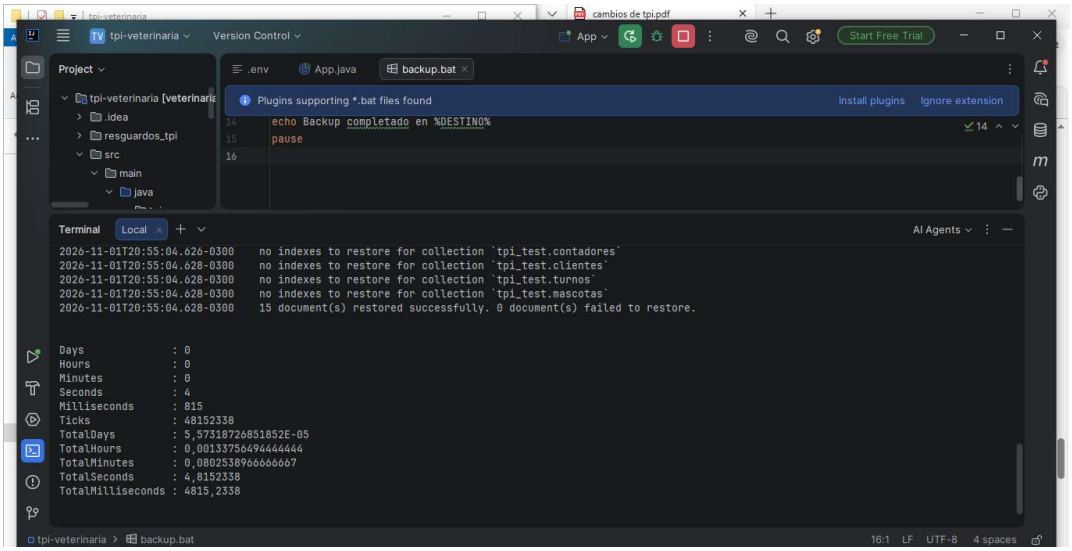
La restauración se probó en una base de prueba (tpi_test), para no pisar la base original, con: mongorestore --uri=<URI> --nsFrom="tpi_veterinaria.*" --nsTo="tpi_test.*" resguardos_tpi\<fecha-hora>. Se restauraron las cuatro colecciones, con 15 documentos y 0 fallos, como se ve en las capturas siguientes.

Análisis RTO / RPO

- RPO (Recovery Point Objective): es la máxima cantidad de datos que se acepta perder. Con un backup ejecutado manualmente una vez al día, el RPO es de hasta 24 horas: se perdería lo cargado desde el último respaldo.
- RTO (Recovery Time Objective): es el tiempo máximo para recuperar el servicio tras una falla. En la prueba realizada, restaurar el backup completo con mongorestore en una base de prueba (tpi_test) tardó aproximadamente 4,8 segundos (medido con Measure-Command), con 15 documentos restaurados y 0 fallos, dado el reducido volumen de datos. Sumando la detección de la falla, disponer de las herramientas y reconectar la aplicación, se estima un RTO total de pocos minutos. En las capturas se ve la generación de la base de prueba y el tiempo de ejecución.







- Mejora propuesta: programar el script con el Programador de tareas de Windows (o cron en Linux) para ejecutarlo automáticamente cada cierto tiempo, lo que reduce el RPO a horas o minutos.

El script necesita tener instaladas las MongoDB Database Tools (mongodump).

7. Credenciales para la corrección

Connection string:
mongodb+srv://emifiscina89_db_user:<CONTRASEÑA>@cluster0.aidqtso.mongodb.net/?appName=Cluster0

Base de datos: tpi_veterinaria

Usuario: emifiscina89_db_user

Contraseña: Emilianoutn1234

El código usa estas credenciales a través del archivo.env, incluido en el ZIP de entrega.

8. Conclusión

El trabajo permitió diseñar un modelo de datos NoSQL aplicando criterios de embedding y linking según el tamaño, el crecimiento y la forma de lectura de cada dato, y llevarlo a la nube con MongoDB Atlas. Se desarrolló una aplicación Java que, como cliente, se comunica con Atlas, que actúa como servidor, mediante el connection string y el driver nativo, y que implementa las operaciones CRUD respetando la baja lógica para conservar el historial.

También se automatizó un mecanismo de backup con mongodump y se analizaron los conceptos de RPO y RTO para entender la continuidad del servicio ante una falla.

Entre las dificultades encontradas estuvieron la configuración del acceso en Atlas (usuario de base de datos y lista de IP permitidas), la instalación de las herramientas de backup y la implementación de identificadores autoincrementales, ya que MongoDB no los ofrece de forma nativa.

Tras la devolución de la cátedra se realizaron mejoras: la actualización quedó restringida a una lista de campos editables con conversión de tipos, las fechas pasaron a guardarse como tipo fecha, el script de backup indica la base a respaldar y controla si el proceso falla, y la entrega incluye todos los archivos necesarios para compilar y ejecutar el proyecto.